CommerceLab Production

Multi-AZ Architecture and Disaster Recovery

Scope	CommerceLab production platform on AWS
AWS account / region	`636830127376` / `eu-central-1` (Frankfurt)
Availability Zones	`eu-central-1a`, `eu-central-1b`, `eu-central-1c`
Kubernetes cluster	`commercelab-prod-cluster` (all brands)
Network	`commercelab-prod-vpc` (`10.30.0.0/16`), private and public subnets in all three AZs

1. Summary

The CommerceLab production platform runs in the AWS region eu-central-1. This region has three Availability Zones (AZs).

Each component in this document runs in two or three AZs. Section 3 gives the AZ layout and the recovery mechanism of each component.

Section 4 gives the sequence of events when one AZ becomes unavailable.

2. Terms

Term	Meaning in this document
AZ	Availability Zone. One or more AWS data centers in a region, with separate power, cooling and network. Frankfurt has three AZs.
Multi-AZ	The component has nodes, copies or network endpoints in two or more AZs.

3. Multi-AZ components

Layer	Component	AZ layout	Recovery mechanism
Network	VPC `commercelab-prod-vpc`	Private and public subnets in 1a, 1b and 1c	Each AZ has its own subnets, so each layer below can place resources in more than one AZ.
Edge	21 Application and Network Load Balancers (brand groups A to E, internal services)	Nodes in 1a, 1b and 1c	AWS sends requests only to healthy load balancer nodes and healthy targets. The DNS name of the load balancer does not change.
Kubernetes control plane	Amazon EKS 1.36, `commercelab-prod-cluster`	AWS-managed, subnets in 1a, 1b and 1c	AWS operates the control plane across AZs. The Kubernetes API stays at the same endpoint.
Worker nodes	Karpenter node pools `ondemand`, `spot`, `spot-2xlarge`	1a and 1b, 58 nodes (320 pods in 1a, 363 pods in 1b)	Kubernetes schedules pods again on healthy nodes. Karpenter starts new nodes in the healthy AZ when more capacity is necessary.
Relational database	Aurora PostgreSQL 14.20, `commercelab-prod-postgresql-cluster`	Writer in 1a (`db.r6g.xlarge`), reader in 1b (Serverless v2)	If the writer stops, Aurora promotes the reader to writer automatically. The cluster endpoint stays the same.
Database proxy	RDS Proxy `prod-postgresql-rdsproxy`	1a and 1b	Applications use one address, `postgresql.commercelab.prod:6432`. After an Aurora failover, the proxy sends connections to the new writer.
Document database	MongoDB replica set, 3 members (`mongodb1`, `mongodb2`, `mongodb3`)	1a and 1b	If the primary member stops, the members elect a new primary automatically. An election needs a majority of the members.

Layer	Component	AZ layout	Recovery mechanism
Message broker	RabbitMQ cluster, 3 nodes, behind `prod-commercelab-rabbitmq-nlb`	Nodes in 1a and 1b, load balancer in 1a, 1b and 1c	The load balancer sends client connections only to healthy nodes. The cluster continues to operate while a majority of the nodes is available.
File storage	41 Amazon EFS file systems (brand files, integration files, WordPress files)	Regional storage, mount targets in 1a, 1b and 1c	EFS keeps the data in more than one AZ. Pods in any AZ mount the same file system.
Object storage	Amazon S3 (media, logs, files)	Regional	S3 keeps each object in a minimum of three AZs.

4. When one AZ becomes unavailable

The platform recovers with the mechanisms in section 3. No manual action starts them. The sequence is:

1. The load balancers stop sending requests to the targets in the unavailable AZ. Requests go to the targets in the healthy AZs.
2. Kubernetes marks the nodes in the unavailable AZ as `NotReady`. After the eviction timeout, it schedules their pods again on nodes in the healthy AZ.
3. Karpenter starts new nodes in the healthy AZ when these pods need more capacity.
4. If the Aurora writer is in the unavailable AZ, Aurora promotes the reader. RDS Proxy sends new connections to the new writer.
5. MongoDB elects a new primary, and the RabbitMQ load balancer sends connections to the healthy nodes, as section 3 describes.
6. The EFS file systems and the S3 data stay available to the pods in the healthy AZ.

Applications with two or more replicas continue to serve during steps 2 and 3. Applications with one replica serve again when their pod starts on the new node.